

VentureMiner AI

Backend & Database Architecture

Document: 05-Backend / 05_backend_database_architecture

Version: v1.0

Date: 2026-07-20

Author: VentureMiner AI Documentation Team

Status: Approved

Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Backend & Database Architecture

Table of Contents

1. Purpose & Scope
2. Backend Principles
3. Service Catalog (recap)
4. API Gateway
5. Service Internal Architecture
 - 5.1 Layered architecture
 - 5.2 Common patterns
6. Persistence Strategy
7. Logical Data Model
8. Physical Data Model (selected)
 - 8.1 `auth.user`
 - 8.2 `workspace.workspace`
 - 8.3 `workspace.membership`
 - 8.4 `opportunity.opportunity`
 - 8.5 `opportunity.opportunity\_event`
 - 8.6 `discovery.discovery\_run`
 - 8.7 `validation.validation\_run`
 - 8.8 `validation.validation\_evidence`
 - 8.9 `scoring.rubric\_version`
 - 8.10 `scoring.score`
 - 8.11 `reporting.report`
 - 8.12 `billing.subscription`
 - 8.13 `audit.audit\_event`
9. Indexes & Performance
 - 9.1 Mandatory indexes
 - 9.2 Vector index
 - 9.3 Hot paths
10. Migrations
11. Multi-Tenancy
 - 11.1 Why RLS over a service-side filter

- 13. Background Jobs & Workflows
- 14. Eventing
 - 14.1 Topics (NATS JetStream)
 - 14.2 Schema registry
- 15. Cross-Service Transactions
- 16. Observability (backend-specific)
- 17. Performance Targets
- 18. Capacity Planning
- 19. Failure Modes
- 20. Appendix
 - 20.1 Glossary
 - 20.2 Revision history
 - 20.3 Cross-references

Backend & Database Architecture

Document 05 — VentureMiner AI The canonical backend and database architecture. Defines service decomposition, data model, query patterns, scaling, and operations for the OLTP plane. The AI plane is detailed in Documents 07–18.

Table of Contents

1. Purpose & Scope
2. Backend Principles
3. Service Catalog (recap)
4. API Gateway
5. Service Internal Architecture
6. Persistence Strategy
7. Logical Data Model
8. Physical Data Model (selected)
9. Indexes & Performance
10. Migrations
11. Multi-Tenancy
12. Caching
13. Background Jobs & Workflows
14. Eventing
15. Cross-Service Transactions
16. Observability (backend-specific)
17. Performance Targets
18. Capacity Planning
19. Failure Modes
20. Appendix

1. Purpose & Scope

This document is the architectural source of truth for the OLTP plane: the user-facing services, persistence, jobs, and observability. The AI plane (Documents 07–18) and the engineering plane (Documents 19–30) are referenced but not detailed here.

2. Backend Principles

1. One service, one data store. No service shares a database.
2. Schema is a contract. Schemas are versioned; breaking changes are MAJOR.

~~3. Events, not shared state. Cross-service state is communicated via NATS JetStream.~~

6. Failure is the default. Every service is designed to be killed mid-request without data loss.

3. Service Catalog (recap)

See Document 02 §4.1 for the full list. Backend services are written in:

- Python (FastAPI) for: auth, workspace, opportunity, discovery, validation, scoring, reporting, agent runtime, rag, memory, plugin, source, search, audit.
- TypeScript (Node + Fastify) for: api-bff, billing, notify, public API (also Python).

4. API Gateway

- Kong (managed) in front of all public traffic.
- Responsibilities:
- TLS termination.
- OIDC validation for browser/SSO flows.
- Rate limiting (per IP, per token, per workspace).
- Request/response logging (sampled, PII-redacted).
- Routing to public REST API or web BFF.

5. Service Internal Architecture

5.1 Layered architecture



5.2 Common patterns

- Repositories abstract SQL; no raw SQL in application code.
- Domain events are emitted on every state change (Document 02 §3.3, CloudEvents).
- Use cases are pure functions of inputs and dependencies — testable without I/O.

OLTP entities	PostgreSQL 16	ACID, JSONB, mature
Vector embeddings	pgvector (v1) → Qdrant (v2)	Single stack v1; scale v2
Cache / rate limits / sessions	Redis 7	Sub-ms latency
Full-text / faceted search	OpenSearch 2.x	Mature, scalable
Object / artifacts	S3-compatible (S3 → R2)	Cost, durability
Event log	NATS JetStream	Durable streams
Workflow state	Temporal	Durable retries
Audit log	Postgres (append-only) + cold storage	Compliance

7. Logical Data Model

ER summary (text, full schema per service).

User 1■■■* Account (OAuth, password)
 User 1■■■* Workspace *■■■1 Owner
 Workspace 1■■■* Membership *■■■1 User
 Workspace 1■■■* Project
 Project 1■■■* Opportunity
 Opportunity 1■■■* DiscoveryHit
 Opportunity 1■■■* ValidationRun
 Opportunity 1■■■* Score
 Opportunity 1■■■* Report
 Opportunity 1■■■* Note
 Opportunity 1■■■* Watch
 Opportunity 1■■■* Alert
 Workspace 1■■■* Rubric 1■■■* RubricVersion
 Workspace 1■■■* Source
 Workspace 1■■■* Integration (Slack, Notion)
 Workspace 1■■■* ApiToken
 Workspace 1■■■* Subscription 1■■■* Invoice
 Workspace 1■■■* AuditEvent
 User 1■■■* MemoryRecord
 DiscoveryRun 1■■■* DiscoveryHit
 ValidationRun 1■■■* ValidationStep
 ValidationStep 1■■■* ValidationEvidence
 Report 1■■■* ReportSection

8. Physical Data Model (selected)

The most important tables, with columns and key constraints.

8.1 auth.user

id	uuid PK	
email	citext UNIQUE	Lowercase, unique
email_verified_at	timestampz	
created_at	timestampz	default now()
disabled_at	timestampz NULL	
last_sign_in_at	timestampz NULL	

8.2 workspace.workspace

id	uuid PK	
name	text	
owner_id	uuid FK → user	
plan	text	Free, Solo, Team, Enterprise
data_region	text	us, eu, apac
created_at	timestampz	
deleted_at	timestampz NULL	Soft delete

8.3 workspace.membership

workspace_id	uuid FK	
user_id	uuid FK	
role	text	owner, admin, member, viewer
joined_at	timestampz	
PRIMARY KEY	(workspace_id, user_id)	

8.4 opportunity.opportunity

id	uuid PK	
workspace_id	uuid FK	RLS-scoped
project_id	uuid FK	
title	text	
summary	text	
status	text	watching, validating, active, archived
score_total	numeric(4,2)	0.00–10.00
score_confidence	text	low, med, high
market_size_usd	bigint	

id	uuid PK	
growth_yoy_pct	numeric(5,2)	
created_at	timestampz	
updated_at	timestampz	
version	int	Optimistic concurrency

8.5 opportunity.opportunity_event

id	uuid PK	
opportunity_id	uuid FK	
type	text	score_changed, validated, archived, etc.
payload	jsonb	
actor_id	uuid	
created_at	timestampz	

8.6 discovery.discovery_run

id	uuid PK	
workspace_id	uuid FK	
seed	text	
window	text	7d, 30d, 90d
depth	text	quick, standard, deep
status	text	queued, running, succeeded, failed, partial
started_at	timestampz	
completed_at	timestampz	
error	text NULL	

8.7 validation.validation_run

id	uuid PK	
opportunity_id	uuid FK	
rubric_version_id	uuid FK	Snapshot of rubric at run time
depth	text	
status	text	
started_at	timestampz	

id	uuid PK	
completed_at	timestampz	

8.8 validation.validation_evidence

id	uuid PK	
step_id	uuid FK	
claim	text	
snippet	text	Quoted text
source_url	text	
source_freshness	timestampz	When the source was captured
confidence	text	low, med, high

8.9 scoring.rubric_version

id	uuid PK	
rubric_id	uuid FK	
version	int	
dimensions	jsonb	Array of
created_at	timestampz	
created_by	uuid	

8.10 scoring.score

id	uuid PK	
opportunity_id	uuid FK	
rubric_version_id	uuid FK	
total	numeric(4,2)	
breakdown	jsonb	Per-dimension
rationale	jsonb	Per-dimension
created_at	timestampz	

8.11 reporting.report

id	uuid PK	
opportunity_id	uuid FK NULL	NULL for comparison reports
type	text	brief, full, compare
title	text	

id	uuid PK	
format_versions	jsonb	{pdf: v1, docx: v1, ...}
status	text	draft, approved, exported
created_by	uuid	
created_at	timestampz	

8.12 billing.subscription

id	uuid PK	
workspace_id	uuid FK UNIQUE	
stripe_subscription_id	text UNIQUE	
plan	text	
status	text	active, past_due, canceled
current_period_start	timestampz	
current_period_end	timestampz	
seats	int	

8.13 audit.audit_event

id	bigserial PK	
workspace_id	uuid	
actor_id	uuid	
action	text	e.g. opportunity.update
resource_type	text	
resource_id	uuid	
before	jsonb	
after	jsonb	
at	timestampz	default now()
ip	inet	
user_agent	text	

Append-only; updates/deletes are denied via DB triggers; the table is partitioned monthly.

9. Indexes & Performance

9.1 Mandatory indexes

opportunity.opportunity	(workspace_id, status, score_total DESC)	Pipeline board
opportunity.opportunity	(workspace_id, project_id, created_at DESC)	Project views
opportunity.opportunity	GIN on to_tsvector('english', title \ summary)	Search
opportunity.opportunity	(workspace_id, updated_at DESC)	Activity
opportunity.opportunity_event	(opportunity_id, created_at DESC)	Activity feed
discovery.discovery_run	(workspace_id, started_at DESC)	List
validation.validation_run	(opportunity_id, started_at DESC)	Detail
validation.validation_evidence	(step_id)	Step detail
scoring.score	(opportunity_id, created_at DESC)	Score history
scoring.rubric_version	(rubric_id, version DESC) UNIQUE	Versioning
billing.subscription	(workspace_id) UNIQUE	One per workspace
audit.audit_event	(workspace_id, at DESC)	Audit log
audit.audit_event	(actor_id, at DESC)	Actor history
auth.user	(email) UNIQUE	Login

9.2 Vector index

- ivfflat on embedding in rag.chunk with lists = sqrt(rows).
- Move to HNSW when row count > 1M.

9.3 Hot paths

- Dashboard → 4 queries (recent activity, watchlist, trends, alerts). All read-replica-routable.
- Pipeline board → 1 query (opportunity with index above).
- Opportunity detail → 4–6 queries (opportunity, latest validation, score, evidence count, related).
- Report viewer → 1 query (report + sections).

10. Migrations

- Tool: Atlas (declarative)

- Long-running migrations (e.g. adding NOT NULL) use a multi-step expand-contract pattern.
- Cutover: expand → deploy code → contract → cleanup.

11. Multi-Tenancy

- Model: shared DB, shared schema, row-level security (RLS).
- Tenant key: workspace_id.
- Enforcement: Postgres RLS policies; every table that holds tenant data has workspace_id NOT NULL and a policy USING (workspace_id = current_setting('app.workspace_id')::uuid).
- Application: every connection sets app.workspace_id per request before executing queries.
- Privileges: the app user is not a superuser; only the migration runner is.

11.1 Why RLS over a service-side filter

- Defense in depth: even a buggy service cannot leak cross-tenant data.
- Simplifies reasoning: data is gated at the DB, not at every query site.

11.2 Hot-path guards

- RLS adds < 0.3ms p99 overhead.
- For very hot reads, we use a denormalized workspace_id column already covered by an index.

12. Caching

Session	Auth	8h idle / 30d max	Sliding
Rate limit counters	API gateway	1m window	TTL
Opportunity hot	Dashboard, list	5m	On update event
Trends	Dashboard	1h	Scheduled rebuild
Source health	Discovery	30s	TTL
Feature flags	Per workspace	5m	TTL

Cache is advisory, never the source of truth. Writes always go to the database first; cache invalidation is event-driven.

13. Background Jobs & Workflows

- Engine: Temporal.
- Use cases: discovery, validation, report generation, periodic reindex, retention sweeps, billing reconciliation.
- Patterns: workflow versioning, signals (e.g. user cancel), queries (current state), retries with backoff, idempotency via workflow ID.

14.1 Topics (NATS JetStream)

opportunity.created	opportunity-svc	search-svc, notify-svc, audit-svc
opportunity.updated	opportunity-svc	search-svc, scoring-svc (re-score trigger)
validation.completed	validation-pipeline	scoring-svc, reporting-svc, notify-svc
score.computed	scoring-svc	opportunity-svc, notify-svc
report.generated	reporting-svc	notify-svc, audit-svc
billing.subscription.changed	billing-svc	auth-svc (entitlements), audit-svc
audit.*	many	audit-svc

14.2 Schema registry

- All events are CloudEvents v1.0.
- Schemas are versioned; a registry service enforces backwards compatibility on producers.

15. Cross-Service Transactions

- No two-phase commit across services.
- Pattern: outbox + idempotent consumer.
- Service writes to its DB and to an outbox table in the same transaction.
- A publisher reads the outbox and emits to NATS.
- Consumers are idempotent (by event ID).
- Compensation: for any business operation that spans services, define an explicit compensation handler.

16. Observability (backend-specific)

- Logs: JSON to stdout; shipped via Vector to Datadog. PII redacted.
- Metrics: Prometheus client exposed on /metrics; scraped by Datadog Agent.
- Traces: OpenTelemetry; 100% sampling for AI plane, 10% for HTTP.
- Health: /healthz (liveness), /readyz (readiness with DB + cache + downstream checks).
- SLOs: see TRD §11.2.

17. Performance Targets

Dashboard load (p75)	< 1.5s
Opportunity list (p75)	< 800ms

Dashboard load (p75)	< 1.5s
Opportunity detail (p75)	< 1.2s
Create opportunity (p95)	< 400ms
Update score (p95)	< 600ms
Search (p75)	< 300ms
Workflow status fetch (p75)	< 200ms

18. Capacity Planning

See TRD §9. Backends follow the same SLO-driven capacity model.

- Postgres: sized for 10k IOPS baseline, burst to 25k. IOPS-bound dashboards scale via read replicas.
- Redis: sized for 10k ops/s at p99 < 1ms.
- OpenSearch: 3 data nodes at launch, scale to 6 by Year 1.
- NATS: 3-node JetStream cluster.

19. Failure Modes

DB primary failure	RDS failover	Multi-AZ, 60–90s
DB replica lag	Replication lag metric	Reroute to primary
Redis down	Health check	Degrade; read-through to DB
OpenSearch down	Health check	Degrade search; show notice
NATS unavailable	Producer metrics	Buffer in outbox; retry
Worker crash	Temporal heartbeat	Restart; replay
Source API down	Per-source error rate	Skip source; flag partial
LLM provider down	Error rate	Fallback provider; surface notice
Schema drift	CI migration check	Block deploy

20. Appendix

20.1 Glossary

OLTP	Online transaction processing
RLS	Row-level security
Outbox	Pattern for reliable cross-service eventing
Idempotency key	Client-supplied key to deduplicate writes

20.2 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

20.3 Cross-references

- TRD: Document 02.
- Application Flow: Document 03.
- AI Architecture: Documents 07–18.
- Engineering: Documents 19–30.

End of Document 05 — Backend & Database Architecture. The schema, indexes, and outbox patterns here are the source of truth for every service in the OLTP plane.